

Python MCP Server Implementation Guide

Overview

This document provides Python-specific best practices and examples for implementing MCP servers using the MCP Python SDK. It covers server setup, tool registration patterns, input validation with Pydantic, error handling, and complete working examples.

Quick Reference

Key Imports

```
python
from mcp.server.fastmcp import FastMCP
from pydantic import BaseModel, Field, field_validator, ConfigDict
from typing import Optional, List, Dict, Any
from enum import Enum
import httpx
```

Server Initialization

```
python
mcp = FastMCP("service_mcp")
```

Tool Registration Pattern

```
python
@mcp.tool(name="tool_name", annotations={...})
async def tool_function(params: InputModel) -> str:
    # Implementation
    pass
```

MCP Python SDK and FastMCP

The official MCP Python SDK provides FastMCP, a high-level framework for building MCP servers. It provides:

- Automatic description and inputSchema generation from function signatures and docstrings

- Pydantic model integration for input validation
- Decorator-based tool registration with `@mcp.tool`

For complete SDK documentation, use WebFetch to load:

<https://raw.githubusercontent.com/modelcontextprotocol/python-sdk/main/README.md>

Server Naming Convention

Python MCP servers must follow this naming pattern:

- **Format:** `{service}_mcp` (lowercase with underscores)
- **Examples:** `github_mcp`, `jira_mcp`, `stripe_mcp`

The name should be:

- General (not tied to specific features)
- Descriptive of the service/API being integrated
- Easy to infer from the task description
- Without version numbers or dates

Tool Implementation

Tool Naming

Use snake_case for tool names (e.g., "search_users", "create_project", "get_channel_info") with clear, action-oriented names.

Avoid Naming Conflicts: Include the service context to prevent overlaps:

- Use "slack_send_message" instead of just "send_message"
- Use "github_create_issue" instead of just "create_issue"
- Use "asana_list_tasks" instead of just "list_tasks"

Tool Structure with FastMCP

Tools are defined using the `@mcp.tool` decorator with Pydantic models for input validation:

```
python
from pydantic import BaseModel, Field, ConfigDict
from mcp.server.fastmcp import FastMCP

# Initialize the MCP server
```

```

mcp = FastMCP("example_mcp")

# Define Pydantic model for input validation
class ServiceToolInput(BaseModel):
    """Input model for service tool operation."""
    model_config = ConfigDict(
        str_strip_whitespace=True, # Auto-strip whitespace from strings
        validate_assignment=True,  # Validate on assignment
        extra='forbid'             # Forbid extra fields
    )

    param1: str = Field(..., description="First parameter description (e.g., 'user123', 'project-abc')",
        min_length=1, max_length=100)
    param2: Optional[int] = Field(default=None, description="Optional integer parameter with
        constraints", ge=0, le=1000)
    tags: Optional[List[str]] = Field(default_factory=list, description="List of tags to apply",
        max_items=10)

@mcp.tool(
    name="service_tool_name",
    annotations={
        "title": "Human-Readable Tool Title",
        "readOnlyHint": True,    # Tool does not modify environment
        "destructiveHint": False, # Tool does not perform destructive operations
        "idempotentHint": True,  # Repeated calls have no additional effect
        "openWorldHint": False   # Tool does not interact with external entities
    }
)
async def service_tool_name(params: ServiceToolInput) -> str:
    """Tool description automatically becomes the 'description' field.

```

This tool performs a specific operation on the service. It validates all inputs using the ServiceToolInput Pydantic model before processing.

Args:

- params (ServiceToolInput): Validated input parameters containing:
 - param1 (str): First parameter description
 - param2 (Optional[int]): Optional parameter with default
 - tags (Optional[List[str]]): List of tags

Returns:

- str: JSON-formatted response containing operation results
- '''
- # Implementation here

pass

Pydantic v2 Key Features

- Use `model_config` instead of nested `Config` class
- Use `field_validator` instead of deprecated `validator`
- Use `model_dump()` instead of deprecated `dict()`
- Validators require `@classmethod` decorator
- Type hints are required for validator methods

python

```
from pydantic import BaseModel, Field, field_validator, ConfigDict
```

```
class CreateUserInput(BaseModel):
    model_config = ConfigDict(
        str_strip_whitespace=True,
        validate_assignment=True
    )

    name: str = Field(..., description="User's full name", min_length=1, max_length=100)
    email: str = Field(..., description="User's email address", pattern=r'^[w\.-]+@[w\.-]+\.\w+$')
    age: int = Field(..., description="User's age", ge=0, le=150)

    @field_validator('email')
    @classmethod
    def validate_email(cls, v: str) -> str:
        if not v.strip():
            raise ValueError("Email cannot be empty")
        return v.lower()
```

Response Format Options

Support multiple output formats for flexibility:

python

```
from enum import Enum
```

```
class ResponseFormat(str, Enum):
    """Output format for tool responses."""
    MARKDOWN = "markdown"
    JSON = "json"
```

```
class UserSearchInput(BaseModel):
```

```

query: str = Field(..., description="Search query")
response_format: ResponseFormat = Field(
    default=ResponseFormat.MARKDOWN,
    description="Output format: 'markdown' for human-readable or 'json' for machine-readable"
)

```

Markdown format:

- Use headers, lists, and formatting for clarity
- Convert timestamps to human-readable format (e.g., "2024-01-15 10:30:00 UTC" instead of epoch)
- Show display names with IDs in parentheses (e.g., "@john.doe (U123456)")
- Omit verbose metadata (e.g., show only one profile image URL, not all sizes)
- Group related information logically

JSON format:

- Return complete, structured data suitable for programmatic processing
- Include all available fields and metadata
- Use consistent field names and types

Pagination Implementation

For tools that list resources:

```

python
class ListInput(BaseModel):
    limit: Optional[int] = Field(default=20, description="Maximum results to return", ge=1, le=100)
    offset: Optional[int] = Field(default=0, description="Number of results to skip for pagination", ge=0)

async def list_items(params: ListInput) -> str:
    # Make API request with pagination
    data = await api_request(limit=params.limit, offset=params.offset)

    # Return pagination info
    response = {
        "total": data["total"],
        "count": len(data["items"]),
        "offset": params.offset,
        "items": data["items"],
        "has_more": data["total"] > params.offset + len(data["items"]),
        "next_offset": params.offset + len(data["items"]) if data["total"] > params.offset + len(data["items"]) else None
    }

```

```
}  
return json.dumps(response, indent=2)
```

Error Handling

Provide clear, actionable error messages:

```
python  
def _handle_api_error(e: Exception) -> str:  
    """Consistent error formatting across all tools."""  
    if isinstance(e, httpx.HTTPStatusError):  
        if e.response.status_code == 404:  
            return "Error: Resource not found. Please check the ID is correct."  
        elif e.response.status_code == 403:  
            return "Error: Permission denied. You don't have access to this resource."  
        elif e.response.status_code == 429:  
            return "Error: Rate limit exceeded. Please wait before making more requests."  
        return f"Error: API request failed with status {e.response.status_code}"  
    elif isinstance(e, httpx.TimeoutException):  
        return "Error: Request timed out. Please try again."  
    return f"Error: Unexpected error occurred: {type(e).__name__}"
```

Shared Utilities

Extract common functionality into reusable functions:

```
python  
# Shared API request function  
async def _make_api_request(endpoint: str, method: str = "GET", **kwargs) -> dict:  
    """Reusable function for all API calls."""  
    async with httpx.AsyncClient() as client:  
        response = await client.request(  
            method,  
            f"{API_BASE_URL}/{endpoint}",  
            timeout=30.0,  
            **kwargs  
        )  
        response.raise_for_status()  
    return response.json()
```

Async/Await Best Practices

Always use async/await for network requests and I/O operations:

python

Good: Async network request

```
async def fetch_data(resource_id: str) -> dict:
    async with httpx.AsyncClient() as client:
        response = await client.get(f"{API_URL}/resource/{resource_id}")
        response.raise_for_status()
        return response.json()
```

Bad: Synchronous request

```
def fetch_data(resource_id: str) -> dict:
    response = requests.get(f"{API_URL}/resource/{resource_id}") # Blocks
    return response.json()
```

Type Hints

Use type hints throughout:

python

```
from typing import Optional, List, Dict, Any
```

```
async def get_user(user_id: str) -> Dict[str, Any]:
    data = await fetch_user(user_id)
    return {"id": data["id"], "name": data["name"]}
```

Tool Docstrings

Every tool must have comprehensive docstrings with explicit type information:

python

```
async def search_users(params: UserSearchInput) -> str:
    """
```

Search for users in the Example system by name, email, or team.

This tool searches across all user profiles in the Example platform, supporting partial matches and various search filters. It does NOT create or modify users, only searches existing ones.

Args:

params (UserSearchInput): Validated input parameters containing:

- query (str): Search string to match against names/emails (e.g., "john", "@example.com", "team:marketing")
- limit (Optional[int]): Maximum results to return, between 1-100 (default: 20)
- offset (Optional[int]): Number of results to skip for pagination (default: 0)

Returns:

str: JSON-formatted string containing search results with the following schema:

Success response:

```
{
  "total": int,      # Total number of matches found
  "count": int,      # Number of results in this response
  "offset": int,     # Current pagination offset
  "users": [
    {
      "id": str,     # User ID (e.g., "U123456789")
      "name": str,   # Full name (e.g., "John Doe")
      "email": str,  # Email address (e.g., "john@example.com")
      "team": str    # Team name (e.g., "Marketing") - optional
    }
  ]
}
```

Error response:

"Error: <error message>" or "No users found matching '<query>'"

Examples:

- Use when: "Find all marketing team members" -> params with query="team:marketing"
- Use when: "Search for John's account" -> params with query="john"
- Don't use when: You need to create a user (use `example_create_user` instead)
- Don't use when: You have a user ID and need full details (use `example_get_user` instead)

Error Handling:

- Input validation errors are handled by Pydantic model
- Returns "Error: Rate limit exceeded" if too many requests (429 status)
- Returns "Error: Invalid API authentication" if API key is invalid (401 status)
- Returns formatted list of results or "No users found matching 'query'"

"""

Complete Example

See below for a complete Python MCP server example:

python

#!/usr/bin/env python3

"""

MCP Server for Example Service.

This server provides tools to interact with Example API, including user search, project management, and data export capabilities.

"""

```
from typing import Optional, List, Dict, Any
from enum import Enum
import httpx
from pydantic import BaseModel, Field, field_validator, ConfigDict
from mcp.server.fastmcp import FastMCP

# Initialize the MCP server
mcp = FastMCP("example_mcp")

# Constants
API_BASE_URL = "https://api.example.com/v1"

# Enums
class ResponseFormat(str, Enum):
    """Output format for tool responses."""
    MARKDOWN = "markdown"
    JSON = "json"

# Pydantic Models for Input Validation
class UserSearchInput(BaseModel):
    """Input model for user search operations."""
    model_config = ConfigDict(
        str_strip_whitespace=True,
        validate_assignment=True
    )

    query: str = Field(..., description="Search string to match against names/emails",
min_length=2, max_length=200)
    limit: Optional[int] = Field(default=20, description="Maximum results to return", ge=1, le=100)
    offset: Optional[int] = Field(default=0, description="Number of results to skip for pagination",
ge=0)
    response_format: ResponseFormat = Field(default=ResponseFormat.MARKDOWN,
description="Output format")

    @field_validator('query')
    @classmethod
    def validate_query(cls, v: str) -> str:
        if not v.strip():
            raise ValueError("Query cannot be empty or whitespace only")
        return v.strip()
```

Shared utility functions

```
async def _make_api_request(endpoint: str, method: str = "GET", **kwargs) -> dict:
```

```
    """Reusable function for all API calls."""
```

```
    async with httpx.AsyncClient() as client:
```

```
        response = await client.request(
```

```
            method,
```

```
            f"{API_BASE_URL}/{endpoint}",
```

```
            timeout=30.0,
```

```
            **kwargs
```

```
        )
```

```
        response.raise_for_status()
```

```
    return response.json()
```

```
def _handle_api_error(e: Exception) -> str:
```

```
    """Consistent error formatting across all tools."""
```

```
    if isinstance(e, httpx.HTTPStatusError):
```

```
        if e.response.status_code == 404:
```

```
            return "Error: Resource not found. Please check the ID is correct."
```

```
        elif e.response.status_code == 403:
```

```
            return "Error: Permission denied. You don't have access to this resource."
```

```
        elif e.response.status_code == 429:
```

```
            return "Error: Rate limit exceeded. Please wait before making more requests."
```

```
        return f"Error: API request failed with status {e.response.status_code}"
```

```
    elif isinstance(e, httpx.TimeoutException):
```

```
        return "Error: Request timed out. Please try again."
```

```
    return f"Error: Unexpected error occurred: {type(e).__name__}"
```

Tool definitions

```
@mcp.tool(
```

```
    name="example_search_users",
```

```
    annotations={
```

```
        "title": "Search Example Users",
```

```
        "readOnlyHint": True,
```

```
        "destructiveHint": False,
```

```
        "idempotentHint": True,
```

```
        "openWorldHint": True
```

```
    }
```

```
)
```

```
async def example_search_users(params: UserSearchInput) -> str:
```

```
    """Search for users in the Example system by name, email, or team.
```

```
    [Full docstring as shown above]
```

```
    """
```

```

try:
    # Make API request using validated parameters
    data = await _make_api_request(
        "users/search",
        params={
            "q": params.query,
            "limit": params.limit,
            "offset": params.offset
        }
    )

    users = data.get("users", [])
    total = data.get("total", 0)

    if not users:
        return f"No users found matching '{params.query}'"

    # Format response based on requested format
    if params.response_format == ResponseFormat.MARKDOWN:
        lines = [f"# User Search Results: '{params.query}'", ""]
        lines.append(f"Found {total} users (showing {len(users)})")
        lines.append("")

        for user in users:
            lines.append(f"## {user['name']} ({user['id']})")
            lines.append(f"- **Email**: {user['email']}")
            if user.get('team'):
                lines.append(f"- **Team**: {user['team']}")
            lines.append("")

        return "\n".join(lines)

    else:
        # Machine-readable JSON format
        import json
        response = {
            "total": total,
            "count": len(users),
            "offset": params.offset,
            "users": users
        }
        return json.dumps(response, indent=2)

except Exception as e:

```

```
        return _handle_api_error(e)

if __name__ == "__main__":
    mcp.run()
```

Advanced FastMCP Features

Context Parameter Injection

FastMCP can automatically inject a `Context` parameter into tools for advanced capabilities like logging, progress reporting, resource reading, and user interaction:

```
python
from mcp.server.fastmcp import FastMCP, Context

mcp = FastMCP("example_mcp")

@mcp.tool()
async def advanced_search(query: str, ctx: Context) -> str:
    """Advanced tool with context access for logging and progress."""

    # Report progress for long operations
    await ctx.report_progress(0.25, "Starting search...")

    # Log information for debugging
    await ctx.log_info("Processing query", {"query": query, "timestamp": datetime.now()})

    # Perform search
    results = await search_api(query)
    await ctx.report_progress(0.75, "Formatting results...")

    # Access server configuration
    server_name = ctx.fastmcp.name

    return format_results(results)

@mcp.tool()
async def interactive_tool(resource_id: str, ctx: Context) -> str:
    """Tool that can request additional input from users."""

    # Request sensitive information when needed
    api_key = await ctx.elicit(
```

```

    prompt="Please provide your API key:",
    input_type="password"
)

```

```

# Use the provided key
return await api_call(resource_id, api_key)

```

Context capabilities:

- `ctx.report_progress(progress, message)` - Report progress for long operations
- `ctx.log_info(message, data)` / `ctx.log_error()` / `ctx.log_debug()` - Logging
- `ctx.elicit(prompt, input_type)` - Request input from users
- `ctx.fastmcp.name` - Access server configuration
- `ctx.read_resource(uri)` - Read MCP resources

Resource Registration

Expose data as resources for efficient, template-based access:

```

python
@mcp.resource("file://documents/{name}")
async def get_document(name: str) -> str:
    """Expose documents as MCP resources.

```

Resources are useful for static or semi-static data that doesn't require complex parameters. They use URI templates for flexible access.

```

"""
    document_path = f"./docs/{name}"
    with open(document_path, "r") as f:
        return f.read()

```

```

@mcp.resource("config://settings/{key}")
async def get_setting(key: str, ctx: Context) -> str:
    """Expose configuration as resources with context."""
    settings = await load_settings()
    return json.dumps(settings.get(key, {}))

```

When to use Resources vs Tools:

- **Resources:** For data access with simple parameters (URI templates)
- **Tools:** For complex operations with validation and business logic

Structured Output Types

FastMCP supports multiple return types beyond strings:

```
python
from typing import TypedDict
from dataclasses import dataclass
from pydantic import BaseModel

# TypedDict for structured returns
class UserData(TypedDict):
    id: str
    name: str
    email: str

@mcp.tool()
async def get_user_typed(user_id: str) -> UserData:
    """Returns structured data - FastMCP handles serialization."""
    return {"id": user_id, "name": "John Doe", "email": "john@example.com"}

# Pydantic models for complex validation
class DetailedUser(BaseModel):
    id: str
    name: str
    email: str
    created_at: datetime
    metadata: Dict[str, Any]

@mcp.tool()
async def get_user_detailed(user_id: str) -> DetailedUser:
    """Returns Pydantic model - automatically generates schema."""
    user = await fetch_user(user_id)
    return DetailedUser(**user)
```

Lifespan Management

Initialize resources that persist across requests:

```
python
from contextlib import asynccontextmanager

@asynccontextmanager
async def app_lifespan():
    """Manage resources that live for the server's lifetime."""
    # Initialize connections, load config, etc.
```

```

db = await connect_to_database()
config = load_configuration()

# Make available to all tools
yield {"db": db, "config": config}

# Cleanup on shutdown
await db.close()

mcp = FastMCP("example_mcp", lifespan=app_lifespan)

@mcp.tool()
async def query_data(query: str, ctx: Context) -> str:
    """Access lifespan resources through context."""
    db = ctx.request_context.lifespan_state["db"]
    results = await db.query(query)
    return format_results(results)

```

Transport Options

FastMCP supports two main transport mechanisms:

```

python
# stdio transport (for local tools) - default
if __name__ == "__main__":
    mcp.run()

# Streamable HTTP transport (for remote servers)
if __name__ == "__main__":
    mcp.run(transport="streamable_http", port=8000)

```

Transport selection:

- **stdio**: Command-line tools, local integrations, subprocess execution
- **Streamable HTTP**: Web services, remote access, multiple clients

Code Best Practices

Code Composability and Reusability

Your implementation **MUST** prioritize composability and code reuse:

1. **Extract Common Functionality:**
 - Create reusable helper functions for operations used across multiple tools
 - Build shared API clients for HTTP requests instead of duplicating code
 - Centralize error handling logic in utility functions
 - Extract business logic into dedicated functions that can be composed
 - Extract shared markdown or JSON field selection & formatting functionality
2. **Avoid Duplication:**
 - NEVER copy-paste similar code between tools
 - If you find yourself writing similar logic twice, extract it into a function
 - Common operations like pagination, filtering, field selection, and formatting should be shared
 - Authentication/authorization logic should be centralized

Python-Specific Best Practices

1. **Use Type Hints:** Always include type annotations for function parameters and return values
2. **Pydantic Models:** Define clear Pydantic models for all input validation
3. **Avoid Manual Validation:** Let Pydantic handle input validation with constraints
4. **Proper Imports:** Group imports (standard library, third-party, local)
5. **Error Handling:** Use specific exception types (`httpx.HTTPStatusError`, not generic `Exception`)
6. **Async Context Managers:** Use `async with` for resources that need cleanup
7. **Constants:** Define module-level constants in `UPPER_CASE`

Quality Checklist

Before finalizing your Python MCP server implementation, ensure:

Strategic Design

- Tools enable complete workflows, not just API endpoint wrappers
- Tool names reflect natural task subdivisions
- Response formats optimize for agent context efficiency
- Human-readable identifiers used where appropriate
- Error messages guide agents toward correct usage

Implementation Quality

- **FOCUSED IMPLEMENTATION:** Most important and valuable tools implemented
- All tools have descriptive names and documentation
- Return types are consistent across similar operations
- Error handling is implemented for all external calls

- Server name follows format: `{service}_mcp`
- All network operations use `async/await`
- Common functionality is extracted into reusable functions
- Error messages are clear, actionable, and educational
- Outputs are properly validated and formatted

Tool Configuration

- All tools implement 'name' and 'annotations' in the decorator
- Annotations correctly set (`readOnlyHint`, `destructiveHint`, `idempotentHint`, `openWorldHint`)
- All tools use Pydantic BaseModel for input validation with `Field()` definitions
- All Pydantic Fields have explicit types and descriptions with constraints
- All tools have comprehensive docstrings with explicit input/output types
- Docstrings include complete schema structure for dict/JSON returns
- Pydantic models handle input validation (no manual validation needed)

Advanced Features (where applicable)

- Context injection used for logging, progress, or elicitation
- Resources registered for appropriate data endpoints
- Lifespan management implemented for persistent connections
- Structured output types used (`TypedDict`, Pydantic models)
- Appropriate transport configured (`stdio` or `streamable HTTP`)

Code Quality

- File includes proper imports including Pydantic imports
- Pagination is properly implemented where applicable
- Filtering options are provided for potentially large result sets
- All `async` functions are properly defined with `async def`
- HTTP client usage follows `async` patterns with proper context managers
- Type hints are used throughout the code
- Constants are defined at module level in `UPPER_CASE`

Testing

- Server runs successfully: `python your_server.py --help`
- All imports resolve correctly
- Sample tool calls work as expected
- Error scenarios handled gracefully